

LECTURE 10

Spring REST & Error Handling

We'll build one small REST API together — and break it on purpose.

Dejan Mitov

UACS · CSCI 2450 · Computer Interface Programming · 2026

Why this lecture?

Every modern app talks over HTTP.

Your phone, your browser, your laptop — all clients of a server somewhere.

Today

Build that server in Spring Boot.

By the end

You will have a working REST API.

Why bother

Your final project lives or dies on this.

We will build a tiny “Friends Directory” service. Same shape as your project.

Learning outcomes

1

Explain

what REST is and what each HTTP verb does.

3

Build

a Spring Boot REST controller from scratch.

5

Search

by writing method names — Spring writes the SQL for you.

2

Map

a CRUD operation onto the right HTTP request.

4

Save

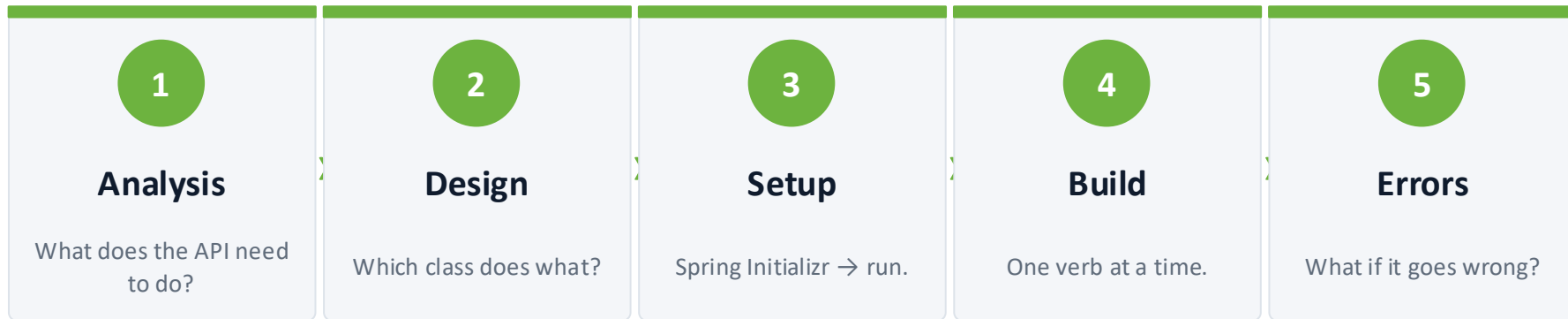
data with a Spring Data JPA repository.

6

Handle

errors so the client gets a useful message, not a stack trace.

Five steps. We'll do them in order.



Same map as your project assignment. Once you do this once, you'll do it every time.

So slow down on each step today — it pays back next time.

STEP 1 / 5

Analysis

What is this thing called REST, and what do we need to build?

Six words to remember

Client

Phone, browser, another server.

Server

Your Spring Boot app, listening on a port.

Request

An HTTP message: a verb + a URL.

Response

Status code + a piece of data.

Resource

The thing you're talking about

JSON

How resources travel on the wire today.

What actually goes over the wire



A request is just: a VERB + a URL + (sometimes) a body in JSON.

A response is: a STATUS CODE + (sometimes) a body in JSON.

That's it. Everything else is just convention.

Learn these. Forget the others for now.

GET

Read a resource.

GET /friends

POST

Create a new resource.

POST /friends

PUT

Update an existing resource.

PUT /friends/42

DELETE

Remove a resource.

DELETE /friends/42

Mnemonic: GET = read · POST = make · PUT = replace · DELETE = remove.

Five numbers to recognise on sight

200	OK	It worked. Here's what you asked for.
201	Created	The new resource exists. POST usually returns this.
400	Bad Request	Your request is malformed. Check what you sent.
404	Not Found	The resource you asked about doesn't exist.
500	Internal Server Error	Something blew up on the server. NOT the client's fault.

Pattern: 2xx = good · 4xx = client's fault · 5xx = your fault.

The verb is already the HTTP method

✗ Don't write URLs like this

GET /getFriend?id=42

POST /createFriend

GET /listAllFriends

POST /updateFriend?id=42

GET /deleteFriendById?id=42

✓ Write them like this

GET /friends/42

POST /friends

GET /friends

PUT /friends/42

DELETE /friends/42

Verbs come from HTTP. URLs name resources. /friends not /listAllFriends.

A tiny “Friends Directory”

The domain

A friend has just three things:

id	Long	Database key
-----------	------	--------------

firstName	String	“Ada”
------------------	--------	-------

lastName	String	“Lovelace”
-----------------	--------	------------

That's it. No relations. No tricky fields.

Same shape as your project's main entity.

What we want to do

- **Register** a new friend
- **List** all friends
- **Find** one friend by id
- **Change** an existing friend
- **Remove** a friend
- **Search** by first / last name

A list of needs becomes a list of endpoints

What we want	HTTP verb	URL	Body?
Register a new friend	POST	/friends	Yes
List all friends	GET	/friends	No
Find one friend by id	GET	/friends/{id}	No
Change an existing friend	PUT	/friends/{id}	Yes
Remove a friend	DELETE	/friends/{id}	No

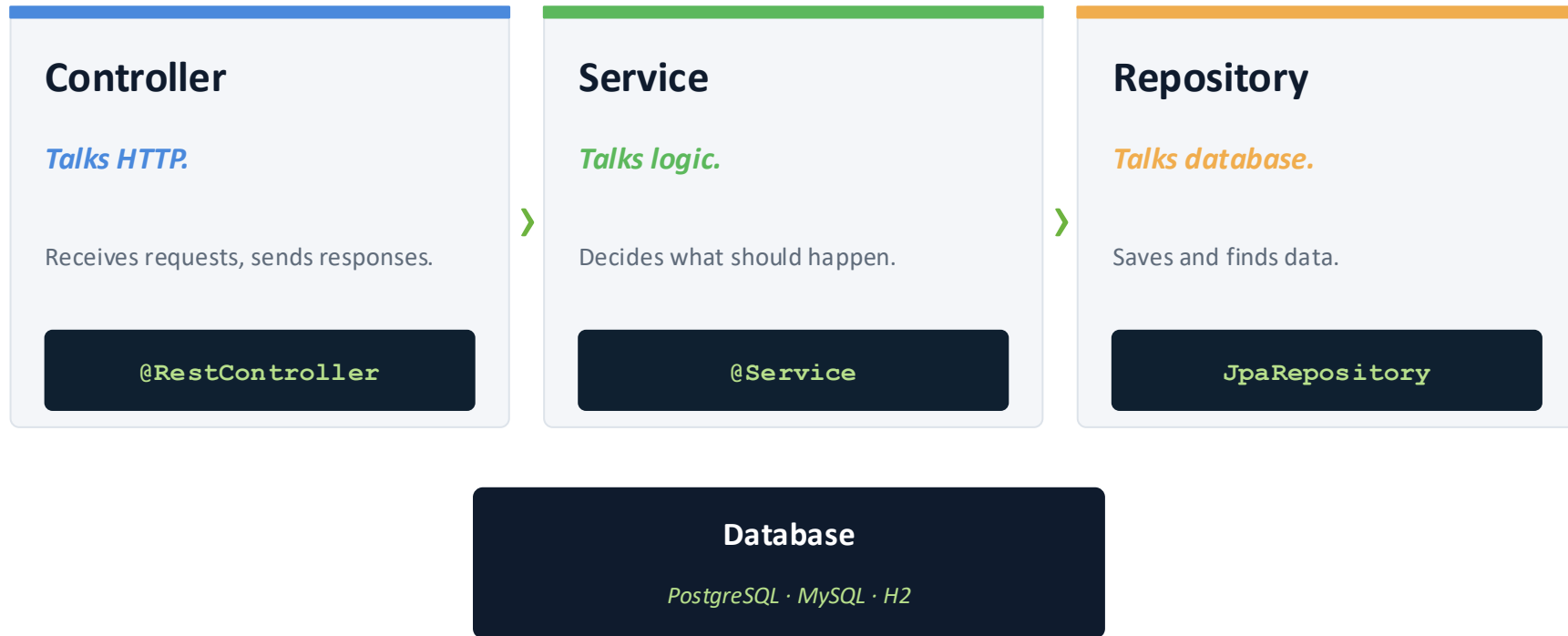
This table is the contract. Once we agree on it, the code writes itself.

STEP 2 / 5

Design

Three small classes, three different jobs.

Each layer has ONE job



Imagine a restaurant.

Waiter

≈ `Controller`

Takes the order, brings the food.
Doesn't cook. Doesn't shop.

Chef

≈ `Service`

Knows the recipe.
Doesn't talk to the customer.

Pantry

≈ `Repository`

Holds the ingredients.
Doesn't know what's being cooked.

Each one is good at its job because it doesn't try to do the others.

Four files, that's all

1	Friend.java	The shape of the data.	@Entity
---	--------------------	------------------------	---------

2	FriendRepository.java	Save & find friends.	interface ... extends JpaRepository<Friend, Long>
---	------------------------------	----------------------	---

3	FriendService.java	Business rules.	@Service
---	---------------------------	-----------------	----------

4	FriendController.java	HTTP endpoints.	@RestController
---	------------------------------	-----------------	-----------------

Spring will wire these together for you. We'll see how — one by one.

Same operation, three method names

Operation	Controller	Service	Repository
Register	<code>create(@RequestBody Friend f)</code>	<code>create(Friend f)</code>	<code>save(f)</code>
List	<code>list()</code>	<code>list()</code>	<code>findAll()</code>
Find	<code>get(@PathVariable Long id)</code>	<code>get(Long id)</code>	<code>findById(id)</code>
Change	<code>update(@PathVariable Long id, @RequestBody Friend f)</code>	<code>update(id, f)</code>	<code>save(f)</code>
Remove	<code>delete(@PathVariable Long id)</code>	<code>delete(Long id)</code>	<code>deleteById(id)</code>

Notice: Repository methods are GIVEN by Spring. We will not write them.

That's the magic of Spring Data JPA — your interface is the implementation.

STEP 3 / 5

Setup

Five minutes from blank screen to running server.

start.spring.io — fill the form

start.spring.io

Project

Maven

Language

Java

Spring Boot

3.x

Java

21

Group

mk.uacs.cip

Artifact

friends

Packaging

Jar

Dependencies — pick three

Spring Web

@RestController + Tomcat

Spring Data JPA

Hibernate + repository magic

H2 Database

Tiny in-memory DB for class

Click GENERATE → unzip → open in IntelliJ. You're ready.

What's in the zip Spring just gave you

```

 src/main/java/mk/uacs/cip/friends/
     FriendsApplication.java      ← your main(). Don't touch yet.
 src/main/resources/
     application.properties      ← config goes here.
 src/test/java/mk/uacs/cip/friends/
     FriendsApplicationTests.java
 pom.xml                      ← dependencies.
 mvnw                        ← Maven wrapper. Run with this.
```

We'll add four new files: Friend.java, FriendRepository.java, FriendService.java, FriendController.java.

Tell Spring about the database

```
# H2 in-memory database — perfect for learning
spring.datasource.url=jdbc:h2:mem:friendsdb
spring.datasource.username=sa
spring.datasource.password=

# Let Hibernate create the table from the @Entity
spring.jpa.hibernate.ddl-auto=update

# Show me the SQL Hibernate runs (handy for class)
spring.jpa.show-sql=true

# Browse the DB at http://localhost:8080/h2-console
spring.h2.console.enabled=true
```

Now run the app:

```
$ ./mvnw spring-boot:run → Tomcat started on port 8080
```

STEP 4 / 5

Build

One verb at a time. Code → run → see it work.

Friend.java — the shape of a friend

```
package mk.uacs.cip.friends;

import jakarta.persistence.*;

@Entity
public class Friend {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;

    // + empty constructor + getters + setters (IntelliJ generates them)
}
```

@Entity says: "Hibernate, please make a table for this." @Id picks the primary key. @GeneratedValue lets the DB assign it.

A QUICK CLARIFICATION

Yes — you still use JPA. It is now packaged under Jakarta EE; that is all that changed for you.

JAKARTA EE *(umbrella platform — formerly Java EE, now hosted by Eclipse Foundation)*

JPA

Java Persistence API

Maps Java objects to database tables.

`@Entity`, `@Id`, `@GeneratedValue`

↑ *what we use today*

Servlet

Handles incoming web requests

Bean Validation

Checks input data is valid

CDI

Wires objects together for you

*Note: a **Bean** is just a Java object that Spring manages for you. **CDI** = Contexts and Dependency Injection.*

WHAT YOU WILL SEE IN CODE

✓ `import jakarta.persistence.*;` ← *Spring Boot 3+, this course*

✗ `import javax.persistence.*;` ← *old tutorials, pre-2022 code*

FriendRepository.java — one line of real code

```
package mk.uacs.cip.friends;

import org.springframework.data.jpa.repository.JpaRepository;

public interface FriendRepository extends JpaRepository<Friend, Long> {

    // Spring will give us:
    //   save(Friend f)           - register or change
    //   findAll()               - list all
    //   findById(Long id)       - find one
    //   deleteById(Long id)    - remove one
    // ...and more, for free.

}
```

That's the repository. We will not write a single SQL query.

FriendService.java — the brain in the middle

```
@Service
public class FriendService {

    private final FriendRepository repo;

    public FriendService(FriendRepository repo) { this.repo = repo; }

    public Friend create(Friend f)           { return repo.save(f); }
    public List<Friend> list()                { return repo.findAll(); }
    public Friend get(Long id)               { return repo.findById(id).orElse(null); }
    public Friend update(Long id, Friend f)  { f.setId(id); return repo.save(f); }
    public void delete(Long id)              { repo.deleteById(id); }
}
```

All the service does is call the repository. Later we'll add real rules — for now, simple is fine.

FriendController.java — the front door

```
@RestController
@RequestMapping("/friends")
public class FriendController {

    private final FriendService svc;

    public FriendController(FriendService svc) { this.svc = svc; }

    // We'll fill in the methods next — one HTTP verb at a time.

}
```

@RestController

Every method returns JSON to the client.

@RequestMapping("/friends")

Every URL in this class starts with /friends.

Register a new friend

```
@PostMapping
public Friend create(
    @RequestBody Friend f) {
    return svc.create(f);
}
```

@RequestBody = parse the JSON body into a Friend object.

Spring picks the right method by the verb (POST) and the URL (/friends).



Postman

POST /friends

Body

```
{
  "firstName": "Ada",
  "lastName": "Lovelace"
}
```

Response

201 Created

```
{
  "id": 1,
  "firstName": "Ada",
  "lastName": "Lovelace"
}
```

List all friends

```
@GetMapping
public List<Friend> list() {
    return svc.list();
}
```

No body, no path variable. Just "give me everything".

Spring serialises the List<Friend> to a JSON array automatically.

Postman

GET

/friends

Response

200 OK

```
[
  { "id": 1, "firstName": "Ada",      "lastName":
    "Lovelace" },
  { "id": 2, "firstName": "Linus",    "lastName":
    "Torvalds" },
  { "id": 3, "firstName": "Charles", "lastName":
    "Babbage" }
]
```

GET /friends/{id}

Find one friend

```
@GetMapping("/{id}")
public Friend get(
    @PathVariable Long id) {
    return svc.get(id);
}
```

@PathVariable = pull the value out of the URL.

/friends/1 → id = 1L

Postman

GET

/friends/1

Response

200 OK

```
{
  "id": 1,
  "firstName": "Ada",
  "lastName": "Lovelace"
}
```

PUT /friends/{id}

Change an existing friend

```
@PutMapping("/{id}")
public Friend update(
    @PathVariable Long id,
    @RequestBody Friend f) {
    return svc.update(id, f);
}
```

Both: id from the URL, new data from the body.

We'll come back to this — there's a bug here we'll find later.

The screenshot shows the Postman interface for a PUT request. The URL is `/friends/1`. The request body is a JSON object: `{ "firstName": "Ada", "lastName": "Byron" }`. The response status is `200 OK`, and the response body is a JSON object: `{ "id": 1, "firstName": "Ada", "lastName": "Byron" }`.

```
Postman

PUT /friends/1

Body
{
  "firstName": "Ada",
  "lastName": "Byron"
}

Response
200 OK

{
  "id": 1,
  "firstName": "Ada",
  "lastName": "Byron"
}
```

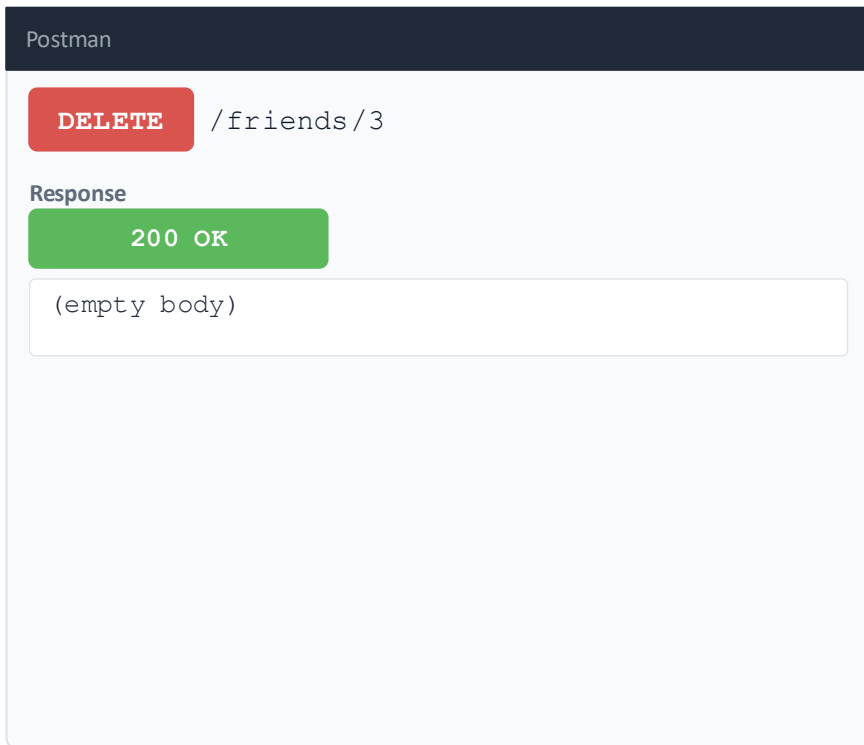
DELETE /friends/{id}

Remove a friend

```
@DeleteMapping("/{id}")
public void delete(@PathVariable Long
id) {
    svc.delete(id);
}
```

`void` return → Spring sends 200 with an empty body.

That's enough for now. We'll learn 204 No Content later.



Five methods. Read top to bottom.

```
@RestController
@RequestMapping("/friends")
public class FriendController {

    private final FriendService svc;
    public FriendController(FriendService svc) { this.svc = svc; }

    @PostMapping
    public Friend create(@RequestBody Friend f) { return svc.create(f); }

    @GetMapping
    public List<Friend> list() { return svc.list(); }

    @GetMapping("/{id}")
    public Friend get(@PathVariable Long id) { return svc.get(id); }

    @PutMapping("/{id}")
    public Friend update(@PathVariable Long id, @RequestBody Friend f) { return svc.update(id, f); }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) { svc.delete(id); }
}
```

Five lines doing five HTTP operations. Notice how little code it really is.

Find friends by name

What we want

Two new endpoints under /friends — one filtered by first name, one by last name.

Example: GET /friends/search?first=Ada → all friends named Ada.

Add to FriendRepository:

```
List<Friend> findByFirstName(String firstName);  
List<Friend> findByLastName(String lastName);  
  
// Look at the method NAMES.  
// Spring reads them and writes the SQL for you.  
// SELECT * FROM friend WHERE first_name = ?
```

Spring's nicest party trick

`findByFirstNameAndLastName`

find

What to do

Return matching rows.

FirstName

Field 1 to match

Looks at the firstName field.

And

Combine

Both must match.

LastName

Field 2 to match

Also looks at lastName.

So `findByFirstNameAndLastName("Ada", "Lovelace")` → `WHERE first_name = ? AND last_name = ?`

Spring reads English-ish method names. Capital letters mark the field boundaries.

One controller method, two query params

```
// In FriendController
@GetMapping("/search")
public List<Friend> search(
    @RequestParam(required = false) String first,
    @RequestParam(required = false) String last) {

    if (first != null && last != null)
        return repo.findByFirstNameAndLastName(first, last);
    if (first != null) return repo.findByFirstName(first);
    if (last != null) return repo.findByLastName(last);
    return repo.findAll();
}
```

required = false → the param is optional. We branch on what was supplied.

Try it in Postman

Postman

GET /friends/search?first=Ada

Response

200 OK

```
[
  { "id": 1, "firstName": "Ada", "lastName": "Lovelace" }
]
```

Postman

GET /friends/search?first=Ada&last=Byron

Response

200 OK

```
[
  { "id": 4, "firstName": "Ada", "lastName": "Byron" }
]
```

STEP 5 / 5

Errors

What if the user does something silly?

Bug → Why → Fix

1

Bug

We send a PUT for a friend that doesn't exist.

2

Why

save() inserts a new row instead of complaining.

3

Fix

Check first. Return 400 if there's nothing to update.

This pattern — bug, why, fix — is how we'll learn error handling.

Don't memorize the rules. Watch the bug appear, then disappear.

PUT to a non-existent id

Step 1. Send PUT to id 999 — which doesn't exist.

Postman	Postman
PUT /friends/999 Body { "firstName": "Ghost", "lastName": "Friend" }	PUT /friends/999 Response 200 OK { "id": 999, ... }

⚠ The response says 200 OK ...

... but we never SAID id 999 existed. The DB just got a new row created with id 999.

That's data pollution. The user thinks they updated. Actually they inserted.

And there's no error message anywhere. Bad.

The repository.save() method is too friendly

```
// In FriendService.update
public Friend update(Long id, Friend f) {
    f.setId(id);
    return repo.save(f); // ← here is the problem
}
```

save() does TWO things at once

- If the id exists → it updates that row.
- If the id doesn't exist → it **INSERTS** a new row with that id.

That second case is the bug. A PUT was meant to update — not invent.

Check first — return ResponseEntity

```
@PostMapping("/{id}")
public ResponseEntity<Friend> update(
    @PathVariable Long id,
    @RequestBody Friend f) {

    if (!repo.existsById(id)) {
        return ResponseEntity.badRequest().build();
    }
    f.setId(id);
    return ResponseEntity.ok(repo.save(f));
}
```

ResponseEntity gives you control over the status code. Now we can say 400 when we mean it.

Same request — different response



Postman

PUT /friends/999

Body

```
{
  "firstName": "Ghost",
  "lastName": "Friend"
}
```

Response

400 Bad Request

(empty body)

What changed?

- Status is now 400, not 200.
- No new row was inserted.
- The client can act on the failure.
- The DB stays clean.

Same input, smarter response.

POST with no body — and we still save it



Postman

POST /friends

Body

```
{}
```

Response

201 Created

```
{
  "id": 5,
  "firstName": null,
  "lastName": null
}
```

⚠ Same problem, different shape

We saved a Friend with NO name. The DB is again polluted.

We could add another `if` check inside the controller — but that gets ugly fast.

Time for a cleaner approach.

Throw an exception — and let one place handle them all.

A custom exception class for this

Step 1. Define a small exception class.

```
public class InvalidFriendException extends RuntimeException {  
    public InvalidFriendException(String msg) { super(msg); }  
}
```

Step 2. Throw it from the service when something is wrong.

```
public Friend create(Friend f) {  
    if (f.getFirstName() == null || f.getLastName() == null) {  
        throw new InvalidFriendException("firstName and lastName are required");  
    }  
    return repo.save(f);  
}
```

Three places, increasing reach

1

Inside the method

try / catch right where it might happen.
Fine for one-offs.

```
try { ... } catch (e) { ... }
```

2

@ExceptionHandler in the controller

One method handles all exceptions thrown by THIS controller.

```
@ExceptionHandler(InvalidFriendException  
.class)
```

3

@ControllerAdvice (global)

One class handles exceptions for ALL controllers.
Best default.

```
@RestControllerAdvice class  
GlobalExceptionHandler { ... }
```

We'll use @ControllerAdvice — same code handles every controller.

One file. All exceptions. All controllers.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

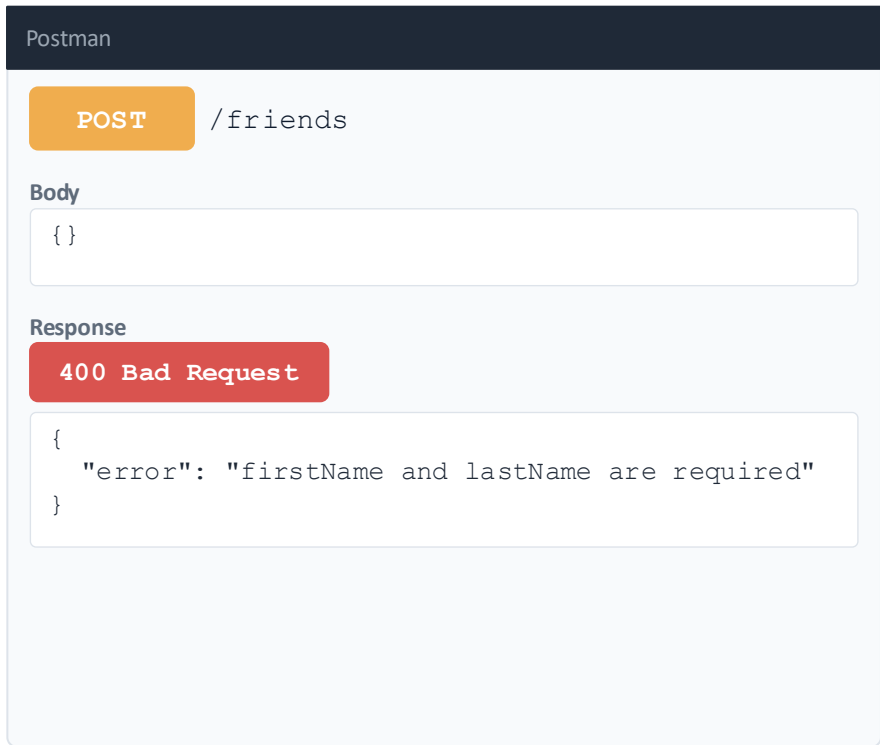
    @ExceptionHandler(InvalidFriendException.class)
    public ResponseEntity<Map<String, String>> handleInvalid(
        InvalidFriendException e) {

        Map<String, String> body = new HashMap<>();
        body.put("error", e.getMessage());

        return ResponseEntity.badRequest().body(body);
    }
}
```

When an InvalidFriendException is thrown — anywhere — this method catches it and turns it into a 400 with JSON.

Same bug, clean response



Postman

POST /friends

Body

```
{ }
```

Response

400 Bad Request

```
{  
  "error": "firstName and lastName are required"  
}
```

What you just saw

- The service decides what's invalid.
- It throws an exception when it is.
- `@ControllerAdvice` catches and translates.
- Client sees 400 + a helpful message.
- DB stays clean.

That's enough for today. There's more — we'll meet it gradually.

The whole map, one more time

1

Analysis

Friend = id + first + last.

2

Design

Controller → Service → Repository.

3

Setup

Initializr + properties + run.

4

Build

5 endpoints + search.

5

Errors

Bug → exception → @ControllerAdvice.

Same five steps for your project's main entity. Same five steps for everything else, ever.

Six things to remember

1 REST = verb + URL + body

Four verbs (GET/POST/PUT/DELETE) and a noun-shaped URL.

3 Repositories are interfaces

Spring writes the implementation — including queries from method names.

5 ResponseEntity for control

Lets you choose the status code — 200, 201, 400, 404.

2 Three layers, one job each

Controller talks HTTP. Service talks logic. Repository talks DB.

4 @RequestBody, @PathVariable, @RequestParam

Three little annotations cover most of what you need.

6 @ControllerAdvice for errors

One place catches them, one place formats the response.

END OF LECTURE 10

Questions?

HOMWORK

Build it yourself

Re-create the Friends Directory from a blank Initializr project.
No copy-paste.

NEXT WEEK

Lecture 11 — Spring Security

Who can call which endpoint, and how to lock down what we just built.